

UF2404EC02_ Feedback ejercicio práctico POO con Python

Maite, en general has hecho una entrega bastante completa y se nota que has entendido los conceptos principales de programación orientada a objetos. Los ejercicios están bien organizados, has comentado bastante el código y, en general, se entiende fácilmente qué estás intentando hacer.

En el **ejercicio 3** lo has resuelto muy bien. Las figuras funcionan correctamente con `imprimir_informe()` sin tener que modificar la función original, y la decisión de hacer que `Cuadrado` herede de `Rectangulo` está muy bien porque así reutilizas `area()` y `perimetro()` en lugar de repetir código. Es una solución sencilla y correcta.

En el **ejercicio 5** también has entendido bien la idea importante: sacar la lógica de los descuentos fuera de `Pedido`. Cada tipo de cliente sabe calcular su propio precio y `Pedido` simplemente delega el cálculo. Además, has añadido `ClienteEstudiante` sin tener que cambiar la clase principal, que era justo lo que se pedía. Como mejora, quitaría la primera versión de `Pedido` que has dejado al principio del archivo, porque después vuelves a crear otra clase con el mismo nombre. No rompe el programa, pero hace el código un poco más confuso de lo necesario.

El **ejercicio 7** está bien resuelto. Has utilizado correctamente la herencia entre `Persona`, `Alumno` y `Profesor`, controlas que solo puedan matricularse alumnos, evitas matrículas repetidas, respetas la capacidad máxima y compruebas que el profesor sea realmente un `Profesor`. El `__str__()` también está bien planteado y muestra la información de manera clara. Como detalle de mejora, podrías validar también que la capacidad máxima sea mayor que cero y revisar alguna pequeña errata en los mensajes.

En el **ejercicio 8** la explicación del MRO está bastante bien. Has entendido que `super()` no significa simplemente “ir al padre”, sino avanzar siguiendo el orden de resolución de métodos. También has calculado correctamente `DBCA` para `D(B, C)` y `DCBA` para `D(C, B)`.

Aquí sí hay un detalle importante: en tu código creas `obj = D()` **antes** de volver a definir la clase `D`. Cuando después escribes una nueva `class D(C, B)`, el objeto `obj` que ya existía sigue perteneciendo a la clase antigua. Por tanto, no cambiaría “automáticamente” a `DCBA`. Tendrías que crear un objeto nuevo después del cambio, por ejemplo `obj2 = D()`, o modificar la definición original y volver a ejecutar el programa desde el principio. La idea que explicas es correcta, pero el código que has dejado no termina de demostrarla.

El **ejercicio 10 es el más completo y está bien planteado**. La clase `Pedido` no necesita saber si está trabajando con un producto físico, digital, una suscripción o un producto con descuento: simplemente llama a `calcular_precio()`. Eso demuestra que has entendido bien el polimorfismo y hace que el sistema sea extensible. También has validado precios, costes de envío, meses y porcentajes de descuento.



UF2404EC02_ Feedback ejercicio práctico POO con Python

Como mejora en este ejercicio, intentaría evitar tener que hacer después `p_descuento.id = "D02"`. Funciona, pero sería más limpio que el identificador se proporcionara correctamente al crear el objeto. También podrías proteger algo más los datos internos del pedido, aunque para lo que pide el ejercicio la solución es perfectamente válida.

En conjunto es una buena entrega. Lo que más destacaría es que utilizas correctamente herencia, composición y polimorfismo y que el código no está montado simplemente para “pasar las pruebas”. Las principales mejoras serían limpiar algo de código redundante y afinar el detalle conceptual del ejercicio 8.



UF2404EC02_ Feedback ejercicio práctico POO con Python

Maite, tu nota provisional es un 7,7/10. Has entregado los cinco ejercicios que correspondían y la selección que has hecho permite llegar hasta un máximo de 8,4 puntos. Además, has incluido los ejercicios 8 y 10, así que cumples también el requisito de realizar al menos uno de los ejercicios de mayor dificultad.

He valorado no solo que los programas funcionen, sino también cómo has planteado las clases, el uso de herencia y polimorfismo, la claridad del código y si se cumplen todos los requisitos del ejercicio. El examen indica precisamente que no basta con que el programa funcione: también se valoran el diseño, la encapsulación, la gestión de errores y la calidad del código.

Desglose de la nota

Ejercicio 3 — 0,8 / 0,8

Este ejercicio está completo y bien resuelto. Has conseguido que `Rectangulo`, `Circulo` y `TrianguloRectangulo` funcionen con `imprimir_informe()` sin modificar la función original, que era una de las condiciones principales.

También has resuelto bien la incorporación de `Cuadrado`: haces que herede de `Rectangulo`, pasando el mismo valor como base y altura. De esta manera reutilizas `area()` y `perimetro()` y evitas repetir código.

Por eso en este ejercicio te doy **la puntuación completa: 0,8/0,8.**

Esto responde exactamente a la petición de añadir `Cuadrado` duplicando la menor cantidad de código posible.

Ejercicio 5 — 0,9 / 1,0

La idea principal está bien entendida. En lugar de que `Pedido` tenga una cadena de `if/elif` para saber qué descuento aplicar, has creado diferentes clases de cliente y cada una sabe calcular su precio.

Lo más importante es que has podido añadir `ClienteEstudiante` sin modificar la nueva clase `Pedido`, que era justamente lo que pedía el ejercicio.

Te descuento una décima principalmente por limpieza de código. Has dejado al principio del archivo la versión antigua de `Pedido` y después vuelves a declarar otra clase `Pedido`. El programa funciona porque la segunda definición sustituye a la primera, pero sería más limpio eliminar el código antiguo una vez hecha la refactorización.

Puntuación: 0,9/1,0.



UF2404EC02_ Feedback ejercicio práctico POO con Python

El objetivo era precisamente poder añadir nuevos descuentos sin modificar la clase que calcula el pedido.

Ejercicio 7 — 1,15 / 1,2

Este ejercicio está prácticamente completo. Has creado correctamente `Persona`, `Alumno`, `Profesor` y `Curso`, y `Alumno` y `Profesor` heredan de `Persona`.

También haces las comprobaciones importantes: solo se pueden matricular objetos `Alumno`, no se puede matricular dos veces al mismo alumno, no se puede superar la capacidad y solo un objeto `Profesor` puede actuar como profesor.

Además, el `__str__()` está bien hecho y permite visualizar de una manera bastante clara el profesor, la capacidad y los alumnos matriculados.

Te descuento únicamente una pequeña parte porque podrías haber añadido alguna validación adicional, por ejemplo comprobar al crear el curso que la capacidad máxima sea un número válido y mayor que cero. También hay alguna pequeña errata en los mensajes de error, pero no afecta al funcionamiento.

Puntuación: 1,15/1,2.

Las condiciones principales requeridas por el ejercicio están implementadas.

Ejercicio 8 — 1,75 / 2,1

Aquí has demostrado que entiendes bastante bien el MRO y el funcionamiento de `super()`. Predices correctamente que con `D(B, C)` el resultado será `DBCA` y explicas correctamente que `super()` sigue el orden establecido por el MRO.

La parte que te resta puntuación está en la modificación del orden de herencia. Es correcto decir que una clase `D(C, B)` tendría un MRO `D → C → B → A → object` y que un objeto creado con esa clase devolvería `DCBA`.

Pero en tu programa creas primero `obj = D()` y después vuelves a definir la clase `D`. Ese objeto `obj` sigue perteneciendo a la primera clase `D`; redefinir después el nombre `D` no transforma el objeto que ya habías creado.

Por tanto, cuando dices que `obj.metodo()` cambia automáticamente a `DCBA`, ahí hay un pequeño error conceptual. Tendrías que crear otro objeto después de redefinir la clase, por ejemplo `obj2 = D()`, o cambiar el orden de herencia y ejecutar nuevamente el programa completo.

Puntuación: 1,75/2,1.



UF2404EC02_ Feedback ejercicio práctico POO con Python

Aquí está la diferencia principal entre tener clara la teoría del MRO y demostrar correctamente el cambio mediante el código. El ejercicio pedía expresamente predecir, comprobar y después modificar el orden de herencia.

Ejercicio 10 — 3,1 / 3,3

Es el ejercicio más completo de la entrega y está muy bien planteado.

Has creado una clase general `Producto` y después las clases `ProductoFisico`, `ProductoDigital` y `Suscripcion`, haciendo que cada producto sea responsable de calcular su propio precio.

Esto permite que `Pedido.calcular_total()` sea muy sencillo: recorre los productos y llama a `calcular_precio()`. No necesita utilizar `if`, `type()` ni saber qué clase concreta tiene delante. Esa es justamente la idea de polimorfismo que buscaba el ejercicio.

`ProductoDescuento` también está bien planteado porque envuelve a otro producto y calcula el descuento partiendo del precio real de ese producto.

También has incluido bastantes validaciones: precios negativos, costes de envío negativos, meses incorrectos, porcentajes de descuento y productos duplicados.

Como mejoras, la colección `productos` queda directamente accesible desde fuera y el cambio manual `p_descuento.id = "D02"` no es demasiado limpio. Sería preferible decidir desde el diseño cómo gestionar el identificador del producto con descuento en lugar de modificarlo después de crear el objeto.

Son detalles pequeños respecto al conjunto del ejercicio, por eso la valoración es alta.

Puntuación: 3,1/3,3.

Es especialmente importante aquí que `calcular_total()` no compruebe el tipo concreto y que admita nuevos productos sin ser modificado, que son dos de las condiciones expresas del ejercicio.



UF2404EC02_ Feedback ejercicio práctico POO con Python

Resultado provisional

Ejercicio	Máximo	Nota
3	0,8	0,8
5	1,0	0,9
7	1,2	1,15
8	2,1	1,75
10	3,3	3,1
TOTAL	8,4	7,7 / 10

En resumen, Maite: es una buena entrega. Todos los programas se ejecutan y, en general, demuestras que has entendido los conceptos que estamos trabajando. Destacaría especialmente los ejercicios 3 y 10. Lo que más te penaliza es el pequeño error conceptual del ejercicio 8 y algunos detalles de limpieza y diseño que todavía pueden mejorarse. La **nota provisional de la parte entregada es 7,7/10**, pendiente de comprobar mediante la defensa escrita que comprendes y puedes modificar las soluciones que has presentado.

Propuesta de defensa escrita para Maite

Ejercicios 8 y 10. El 8 es precisamente donde aparece la duda conceptual y el 10 es el ejercicio más completo. Así la defensa sirve realmente para confirmar qué entiendes y no simplemente para repetir lo que ya funciona. El examen contempla que la defensa valore la capacidad de explicar, predecir y modificar el código.

